PRODUCTION ML CHURN PREDICTION

Technical report | Evaluation, deployment artifacts, serving, and operations

Project goal	Predict whether a customer will be inactive over the next 30 days.
Target	`inactive_next_30d`
Serving	FastAPI, packaged scikit-learn model, Prometheus metrics.
Operations	Docker Compose, Grafana, Alertmanager, offline drift analysis, manual release/rollback.

Scope and evidence note. The dataset is synthetic. Metrics describe experiments on this generated dataset and do not establish real-world business lift or live production performance. This is a portfolio implementation of engineering patterns, not a claim of a live production deployment.

Executive summary

The workflow builds weekly customer snapshots, engineers historical features, applies a time-aware train/validation/test split with a 30-day label-window embargo, compares Logistic Regression, Random Forest, and XGBoost, tracks experiments with MLflow, and serves a selected model bundle through FastAPI.

Random Forest remains the current champion because XGBoost did not meet the configured meaningful-improvement gate. The project has also been hardened with deployment-artifact metadata and checksum tests, explicit local Grafana credentials, and a manual release/rollback runbook. These checks improve reproducibility; they do not replace environment-specific operational validation.

1. Problem, data, and evaluation design

The binary target is `inactive_next_30d`: 1 indicates inactivity over the following 30 days under the dataset's label construction; 0 indicates active. Features are constructed using information available up to each prediction timestamp, while labels use future activity.

Dataset property	Value
Customer-snapshot rows	3,239,195
Unique customers	48,704
Weekly snapshots	126
Snapshot range	2024-04-01 to 2026-08-24
Positive label rate	Approximately 14.4%
Processed dataset columns	82, including IDs, timestamp, and target
Train / validation / test rows	1,654,654 / 341,513 / 939,500
Held-out test period	2026-04-06 to 2026-08-24

Snapshots are keyed by (customer_id, prediction_timestamp); validation found no duplicate keys in the inspected processed dataset. Customers may appear in multiple snapshots, so this is temporal evaluation rather than customer-disjoint evaluation. A 30-day embargo is used to reduce overlap between label windows across splits. The test period is reserved for final evaluation and descriptive time-window analysis rather than repeated tuning.

Source entities represented in the synthetic data include customers, orders, payments, support tickets, subscriptions, marketing events, and customer feedback.

2. Model evaluation

Validation comparison

Model	ROC-AUC	PR-AUC	Brier ↓
Logistic Regression	0.686180	0.238063	0.116942
Random Forest	0.687386	0.238627	0.116970
XGBoost	0.686406	0.235758	0.117036

Random Forest is retained as champion. XGBoost passed configured safety gates but did not achieve the meaningful-improvement criterion. Differences are small; this is a conservative gate-based decision, not a claim of broad model superiority.

Held-out Random Forest test metrics

Metric	Value
ROC-AUC	0.690006
PR-AUC (average precision)	0.237343
Brier score	0.115705

ROC-AUC measures ranking quality; PR-AUC is informative for an imbalanced target; Brier score measures probability prediction error, with lower values better. These results are conditional on the synthetic data and label-generation process.

3. System architecture and data flow

The following logical flow summarizes the repository components. It describes the implemented portfolio workflow, not an automated production deployment path.

1. Data construction
Synthetic raw entities → weekly snapshots → engineered features and labels

↓

2. Evaluation
Schema/data checks → temporal split + embargo → model training and validation

↓

3. Model governance
MLflow tracking/registration → validation-only champion/candidate gates

↓

4. Deployment bundle
`random_forest_v1.joblib` + `model_metadata.json`

↓

5. Inference
FastAPI endpoints: `/health`, `/predict`, `/metrics`

↓

6. Operations
Prometheus metrics and alert rules; Grafana dashboards; Alertmanager; offline drift analysis

The repository separates the longer data/training workflow from the clean-clone inference path. A clone can start the API using the committed deployment bundle, provided the runtime dependencies and artifacts are present; regenerating data and retraining may require generated local data and MLflow state.

4. Deployment artifact integrity

The committed deployment bundle consists of `artifacts/deployment/random_forest_v1.joblib` and `artifacts/deployment/model_metadata.json`. Metadata records the model name/version, MLflow URI, target, ordered feature list, feature count, class labels, filename, and SHA-256 checksum.

The deployment-artifact tests validate:

- Required metadata fields and the expected 78-feature contract.
- Unique, ordered feature names and consistency with the deployed pipeline.
- SHA-256 checksum agreement between the model file and metadata.
- A deterministic synthetic prediction smoke test with finite, bounded probabilities.

These checks run independently of a local MLflow database. When intentionally replacing the model, the artifact and metadata must be updated together and reviewed as one release. A checksum confirms file identity against metadata; it does not prove model quality.

5. API and local service stack

The FastAPI service in `src/api/app.py` exposes `GET /health`, `POST /predict`, and `GET /metrics`. The prediction endpoint validates the feature contract and returns an inactivity probability for a supplied feature record.

Docker Compose runs the API, Prometheus, Grafana, and Alertmanager. Grafana credentials are configured through local environment variables rather than a hardcoded password. The local `.env` file should be created from `.env.example`, set to a unique password, and kept out of version control.

Persistent Grafana state may retain credentials from initial volume setup; changing the environment variable does not necessarily reset an already-initialized account. Avoid deleting persistent volumes as a routine troubleshooting step.

6. CI, reproducibility, and release controls

A clean clone has two distinct usage paths:

Inference path: install Python 3.11 dependencies, use the committed deployment bundle, start the FastAPI app, and run API/artifact tests.

Rebuild path: generate snapshots, build features, validate data, create temporal splits, train/evaluate models, and use local MLflow tracking as required by the scripts.

The CI workflow checks repository test fixtures and deployment artifacts, runs API tests, runs temporal-boundary tests with its configured exclusion for a generated-data-dependent invariant, runs deployment artifact integrity tests when configured, and builds the Docker image. Consult the current workflow file for the exact executed steps.

Recommended local verification commands

```
python -m pytest -q
docker compose config --quiet
docker build -t customer-inactivity-api:local .
git diff --check
```

7. Release and rollback runbook

Before release, review validation/test results, feature schema, model identity, artifact checksum, API behavior, and rollback readiness. Record the Git commit SHA, model version, checksum, evaluation report, test results, and release notes.

Local release verification

```
docker compose up -d --build
 docker compose ps
 Invoke-RestMethod http://localhost:8000/health
 docker compose logs --tail 100 api
```

Verify the health response, expected model version and feature count, prediction behavior, and metrics availability. A healthy endpoint confirms service health; it does not prove model quality.

Manual rollback outline

1. Identify the last known-good commit and deployment bundle.
2. Restore the known-good model and metadata files from that commit without moving the current branch.
3. Run artifact-integrity and API tests.
4. Commit the rollback as a new change, rebuild/restart the API, and verify health and predictions.
5. Record the reason, root cause, rollback commit, and follow-up actions.

Example command (replace the placeholder with a real commit SHA):

```
git restore --source=<known-good-commit> -- artifacts/deployment/random_forest_v1.joblib artifacts/deployment/model_metadata.jso
```

Restoring the local deployment bundle does not restore the MLflow tracking database. The documented release/rollback flow is manual; the project does not claim automated model promotion, deployment, rollback, or retraining.

8. Monitoring and limitations

The repository includes API metrics for Prometheus, alert rules, Alertmanager configuration, Grafana for local dashboards, and an offline drift monitor. The drift monitor compares a reference period with a later period using Population Stability Index for features and prediction scores, and writes a JSON report when its required inputs are available.

Drift is a diagnostic signal, not proof of failure. Investigate data quality, feature pipeline changes, concept drift, and customer behavior before acting. Historical ledger metrics (ROC-AUC 0.75162, PR-AUC 0.35004, Brier 0.11318 for a 5,000-row demonstration) are not directly comparable to held-out test metrics because the temporal composition differs.

Key limitations

- Synthetic data cannot establish real customer behavior or business impact.
- Customers recur across snapshots; the split is temporal rather than customer-disjoint.
- Rolling windows overlap and are not independent estimates.
- PSI thresholds and promotion gates are engineering defaults that need business-specific calibration.
- Offline drift reports are not a live drift-triggered retraining system.
- The project demonstrates a portfolio workflow, not a live production deployment.

9. Repository hygiene

Before committing, run tests and configuration checks, review staged and unstaged diffs, and ensure secrets, local `.env`, generated datasets, local MLflow databases, and logs are not accidentally tracked. The tracked deployment bundle is intentional; generated experiment and training outputs should follow the repository's ignore policy.

Conclusion

This project demonstrates disciplined temporal evaluation, conservative model promotion, deployment artifact integrity, API serving, local observability, and a documented manual recovery path. Its strongest portfolio signal is the end-to-end engineering workflow and explicit limitations—not an unsupported claim of production business performance.